Lab 5 Report

Jatin Punjabi and Jeehan Zaman, EC413

1 Overview

We built a 4-stage, 32-bit pipelined datapath that executes R-type and I-type instructions, following the figure in the handout. Instruction bits 30 and 29 act as a valid (NOP) bit and an immediate-select bit, and bits 28 to 26 go directly to the ALU. The ALU is the provided `Ideal_ALU` and supports MOV, NOT, ADD, SUB, OR, AND and the extra-credit signed SLT. Register names carry a stage prefix (`S1_`, `S2_`, `S3_`), and the design uses only the provided `DFF`, `nbit_reg`, `nbit_mux`, `nbit_demux` and `nbit_register_file` modules plus one small decoder. Data hazards are avoided in software by inserting NOPs, as the handout allows.

ALUOp	**Function**	**Result**
000	MOV	$R1 = R2$
001	NOT	$R1 = \sim R2$
010	ADD	$R1 = R2 + R3$
011	SUB	$R1 = R2 - R3$
100	OR	$R1 = R2 \mid R3$
101	AND	$R1 = R2 \,\&\, R3$
110	SLT	$R1 = 1$ if $R2 < R3$ (signed), else 0

Bits	**Field**	**Meaning**
31	unused	ignored
30	valid / WriteEnable	1 = instruction, 0 = NOP (never writes)
29	DataSrc	0 = register operand, 1 = 16-bit immediate
28:26	ALUOp	ALU function
25:21	r1	destination register (WriteSelect)
20:16	r2	first source register (ReadSelect1)
15:11	r3	second source register (ReadSelect2), R-type
15:0	imm	immediate, zero-extended, I-type

2 Design hierarchy

```
Pipeline
|-- Decode                    (field slicing, combinational)
|-- nbit_reg x 7              Stage 1 registers (S1_)
|-- nbit_register_file        (Stage 2 read, write-back from Stage 3)
|   |-- nbit_demux            (write select)
```

```
|   `-- DFF x 32 x 32
|-- nbit_reg x 7              Stage 2 registers (S2_)
|-- nbit_mux x 32             (operand select, one per bit)
|-- Ideal_ALU                 (Stage 3)
`-- nbit_reg x 3              Stage 3 registers (S3_)
```

Testbenches: `Pipeline_test`, which runs the handout's sample program through the whole datapath, and `Pipeline_tb`, a self-checking testbench with a behavioral model that covers every ALU function, corner cases, NOPs, random programs and reset. The register file and the ALU are tested through both, as described below.

3 Module descriptions

`Decode`. Combinational wiring that slices the 32-bit instruction into `WriteEnable` (bit 30), `DataSrc` (bit 29), `ALUOp` (bits 28:26), `WriteSelect` (25:21), `ReadSelect1` (20:16), `ReadSelect2` (15:11) and `Imm` (15:0). Keeping the instruction format in one module keeps the top level simple.

`Pipeline`. The top level, with ports `InstrIn`, `Reset`, `Clk`, `WriteEnable` and `ALUOut`.

- **Stage 1.** The decoded fields are latched in `S1_RS1`, `S1_RS2`, `S1_WS`, `S1_Imm`, `S1_ALUOp`, `S1_DataSrc` and `S1_WE`. The global `WriteEnable` input is ANDed with the instruction's valid bit before it is latched.
- **Stage 2.** The register file reads the S1 select lines. The two read values, together with the immediate, write select, ALU operation, data source and write enable passed on from S1, are latched into the S2 registers.
- **Stage 3.** A row of 32 one-bit `nbit_mux` instances (generate loop) chooses the second ALU operand: the S2 read data 2 when `DataSrc` is 0, or the zero-extended immediate when it is 1. The first operand is S2 read data 1. The ALU result, write select and write enable are latched in `S3_ALUOut`, `S3_WS` and `S3_WE`.
- **Write-back.** `S3_ALUOut`, `S3_WS` and `S3_WE` drive the register file's write port, and `S3_ALUOut` is also the output `ALUOut`.

Provided modules. `nbit_reg` (parameterized register built from `DFF`s, with synchronous reset) is used for every pipeline register. `nbit_register_file` has two combinational read ports and a clocked write port. `nbit_mux` is a one-bit multiplexer, used here as a 2-to-1 select. `Ideal_ALU` is the provided behavioral ALU.

Why WriteSelect and WriteEnable go through the pipeline. By the time an instruction reaches write-back, `InstrIn` is already holding a later instruction (three instructions later in the steady state). If the register file took its write controls directly from `InstrIn`, the result of the older instruction would be written into the destination register of the newer one, and a NOP could end up enabling or blocking someone else's write. Passing `WriteSelect` and `WriteEnable` down the stages together with the data keeps each result paired with its own destination and its own NOP bit.

4 Test setup

We use two testbenches. `Pipeline_test` resets the design for 100 ns, then applies the handout's six-instruction sequence, one instruction per 10 ns clock, followed by four NOPs so the last instruction completes. Instructions are applied on the falling clock edge so they are stable at the next rising edge. The sequence includes one extra ADDI as a hazard spacer:

#	Instruction	Expected result
1	ADDI r1, 0x0A	r1 = `0000000A`
2	ORI r2, 0x02	r2 = `00000002`
3	ADDI r3, 0x07	r3 = `00000007`
4	ADDI r6, 0xFF	r6 = `000000FF`
5	AND r4 = r1 & r2	r4 = `00000002`
6	SUB r5 = r1 − r3	r5 = `00000003`

After the NOPs, the testbench compares registers r1 to r6 inside the register file to the expected values (from the prelab trace) and prints `PASS` or `FAIL` for each. All six passed. The waveform in Figure 1 is from this run, and the sections below use it.

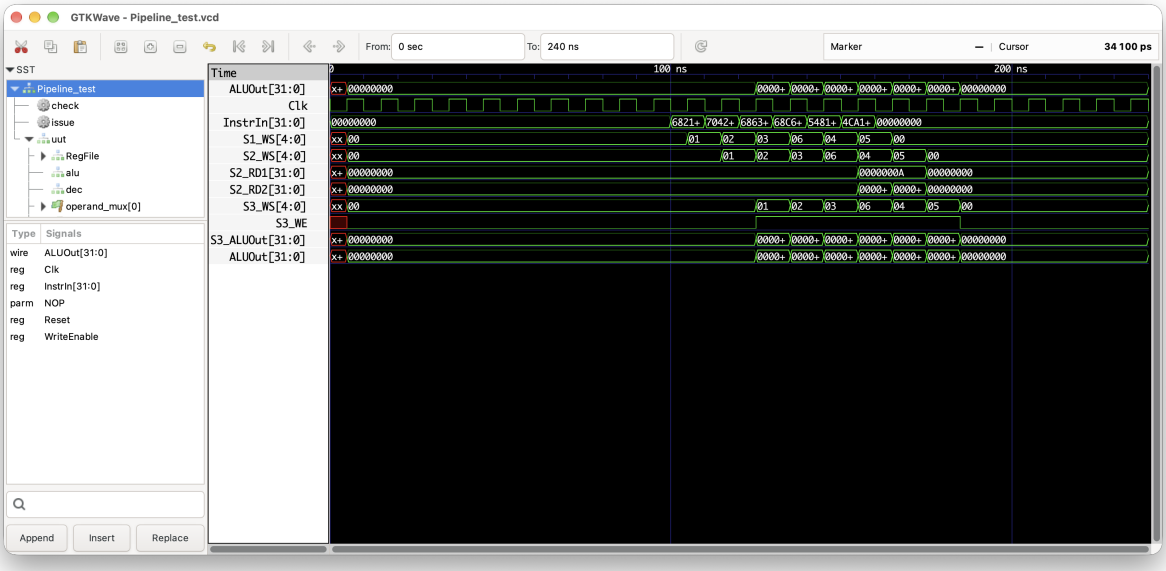


Figure 1: Pipeline timing for the sample program: instructions enter one per clock and results leave `ALUOut` one per clock, three edges later.

Self-checking testbench

`Pipeline_tb` compares the design against a behavioral model: a 32-entry register array and an ALU function written separately from `Ideal_ALU`. Each instruction is applied on the falling clock edge and followed by NOPs, so it completes before the next one starts and no data hazards occur. When an instruction reaches Stage 3, `ALUOut` is checked against the model. One edge later, after write-back, all 32 registers are compared with the model. The corner operands are built by the

program itself: 0, 1, FFFFFFFF, 2, 80000000 (by doubling 1 thirty-one times) and 7FFFFFFF (by subtracting 1 from 80000000). The test then runs:

- **Corner cases, R-type.** Every pair of corner registers for all seven functions.
- **Corner cases, I-type.** Every corner register with every corner immediate (0000, 0001, FFFF, 7FFF, 8000, 0002) for all seven functions.
- **NOPs.** Instructions with bit 30 clear but otherwise valid fields must leave the register file unchanged.
- **Global WriteEnable low.** Valid instructions must not write; a write succeeds again once WriteEnable is raised.
- **Random programs.** 500 random instructions (either type, any function, any registers).
- **Reset.** The register file and ALUOut return to zero.

The run performed 35,145 checks with 0 errors. To confirm that the testbench can detect faults, we forced the Stage 3 write enable high; the run then reported 2,663 errors.

5 Register file testing

The register file is tested through the pipeline. The writes go in through the Stage 3 write-back port, and the reads go out through the two read ports into the Stage 2 registers. In Figure 1:

- **Writes.** Each instruction's result appears on S3_ALUOut and its destination on S3_WS (01, 02, 03, 06, 04, 05) with S3_WE high. The register file writes on the next rising edge. For example, r1 receives 0000000A at the 135 ns edge.
- **Reads on both ports.** Instruction 5 (AND r4 = r1 & r2) reads r1 and r2, and S2_RD1 and S2_RD2 show 0000000A and 00000002 at 155 ns. Instruction 6 (SUB) reads r1 and r3 and shows 0000000A and 00000007 at 165 ns. This shows that values written earlier are read back correctly on both ports.
- **Write timing.** A value is not readable until the edge after it is written. This is why a reader must be at least three slots behind its writer (see the hazard discussion below).
- **WriteEnable low.** After the last instruction, the NOPs have S3_WE low and nothing is written. The final check of r1 to r6 passes after the NOPs, so they were not disturbed.
- **Reset.** Reset is high for the first 100 ns, and all stage registers and the register file start at 0 (the read ports show 00000000 before the first instruction).
- **Exhaustive check.** Pipeline_tb compares all 32 registers with the model after every instruction, so any write to the wrong register, any missed write and any write by a NOP would be reported. It also confirms that reset clears every register.

6 ALU testing

The ALU (Ideal_ALU) is tested through the pipeline as well. The Stage 3 operand MUX gives it either a register value or the zero-extended immediate, and the result is latched in S3_ALUOut/ALUOut. In Figure 1:

- **ADD with immediate (ADDI).** Instructions 1, 3 and 4 add an immediate to a register that is 0, and ALUOut shows 0A, 07, FF. This also tests that DataSrc = 1 selects the immediate.
- **OR with immediate (ORI).** Instruction 2 gives 02.
- **AND with registers.** Instruction 5 (DataSrc = 0) takes 0A and 02 from the register file and gives 02.
- **SUB with registers.** Instruction 6 takes 0A and 07 and gives 03.

- **NOP.** After the last instruction, `InstrIn` is 0 (bit 30 = 0). `ALUOut` returns to `00000000` at 185 ns because a NOP reads r0 (still 0) and computes MOV, and `S3_WE` stays low so the register file is not modified.
- **All functions, including SLT.** `Pipeline_tb` exercises MOV, NOT, ADD, SUB, OR, AND and signed SLT with both register and immediate operands, using the corner values `0`, `1`, `FFFFFFFF`, `7FFFFFFF` and `80000000`. The signed boundary cases (`7FFFFFFF` against `80000000`) are covered by the register-register pairs, since the zero-extended immediate can never be negative. All results match the model.

All of these results match the expected values in the table above.

7 Datapath and timing diagrams

Figure 1 (95–205 ns): The first instruction, `6821000A` (ADDI r1, 0x0A), is applied at 100 ns. At the 105 ns rising edge its destination (r1) appears in `S1_WS`, at 115 ns in `S2_WS`, and at 125 ns in `S3_WS`, where its result `0000000A` also appears on `ALUOut`. The register file writes r1 at the next edge (135 ns). Each later instruction follows ten nanoseconds behind, so four different instructions are in flight at once (one per stage). The results `0A, 02, 07, FF` appear in `ALUOut` at 125, 135, 145 and 155 ns, then `02` (AND) at 165 ns and `03` (SUB) at 175 ns. This is one new result per clock once the pipeline is full, with a latency of three clock edges from instruction to `ALUOut`.

Data hazards

The datapath has no forwarding or stalling. A result is written into the register file at the fourth clock edge after its instruction is applied, so an instruction that reads the same register must be at least three instruction slots behind the one that writes it; otherwise it reads the old value. This is why the sample program contains the extra ADDI (instruction 4): without it, the AND (instruction 5) would be only two slots behind the ORI that writes r2. In the final program the tightest dependencies are exactly three slots (instruction 2 to 5 and instruction 3 to 6), and all results are correct. `Pipeline_tb` avoids hazards by following every instruction with NOPs.